

Parallel Programming Assignment Reports

Students Information

علي عصام عبد الحليم علي

عبدالله محمد أحمد محمد

عبد الرحمن عواد جاد عبود

عبدالله عمر علي سيد

علي عماد علي عبداللاه

احمد صبحي لطفي

#Report 1

Problem Information

Problem Title: Pi with a loop and a reduction

Lecture Number: Unit 2: The core features of OpenMP- Disc 4: Pi program wrap-up

Problem Description

calculate the value of Pi using numerical integration and improve the program's performance using parallel programming

Parallelization Approach

Pi with a loop and a reduction

Solution Explanation

Each thread computes the value of the function for its assigned steps and adds the results to a local sum. After all threads finish, the partial sums are combined using OpenMP's reduction clause. Finally, the result is multiplied by the step size to obtain the approximation of PI.

Source Code

```
#include <iostream>
#include <omp.h>
double step;
static long num_steps=100000;
using namespace std;
int main()
{
    int i;
    double x, pi, sum=0;
    step = 1 / (double)num_steps;
#pragma omp parallel
    {
        double x;
#pragma omp for reduction(+:sum)
        for (int i = 0; i < num_steps; i++) {
            x = (i + 0.5) * step;
            sum = sum + 4.0 / (1.0 + x * x);
        }
    }

    pi = step * sum;
    cout << pi;
}
```

Execution Results

```
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>g++ -fopenmp parallel.cpp
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>.\a.exe
3.14159
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>|
```

Conclusion

OpenMP can be used to parallelize a computational task. By dividing the work among multiple threads and using reduction for synchronization.

#Report 2

Problem Information

Problem Title: Using a critical section to remove impact of false sharing

Lecture Number: Unit 2: The core features of OpenMP - Disc 3: Synchronization overhead and eliminating false sharing

Problem Description

calculate the value of Pi using numerical integration and improve the program's performance using parallel programming

Parallelization Approach

The program uses OpenMP to create multiple threads that work simultaneously.

The iterations of the loop are divided among the available threads. Each thread computes a local partial sum for its assigned iterations. After completing its calculations, each thread enters a **critical section** to safely add its partial result to the shared variable representing the value of π .

Solution Explanation

After creating the OpenMP threads, each thread determines its thread ID and the total number of threads. The iterations are distributed manually such that each thread processes every N-th iteration, where N is the total number of threads.

Once all iterations are completed, each thread enters a **critical section** and adds its local sum to the global approximation of PI.

Finally, the computed value of PI is printed.

Source Code

```
int main()
{
    double pi = 0.0; int nthreads = 0;
    step = 1.0 / num_steps;
    omp_set_num_threads(2);
#pragma omp parallel
    {
        int thread_id, total_threads, int i;
        double x, partial_sum = 0.0;
        thread_id = omp_get_thread_num();
        total_threads = omp_get_num_threads();
        if (thread_id == 0)
            nthreads = total_threads;

        /* Each thread processes its own chunk */
        for (i = thread_id; i < num_steps; i += nthreads)
        {
            x = (i + 0.5) * step;
            partial_sum += 4.0 / (1.0 + x * x);
        }

        /* update global result */
#pragma omp critical
        {
            pi += partial_sum * step;
        }
    }
    cout << "pi: " << pi;
    return 0;
}
```

Execution Results

```
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>.\a.exe  
pi: 3.14159  
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>|
```

Conclusion

This assignment demonstrated the use of OpenMP for parallel programming in C. The numerical integration method was successfully parallelized by distributing iterations among multiple threads. The use of synchronization through a critical section ensured correct computation of the shared result. The experiment showed how parallel programming can improve the efficiency of computationally intensive tasks while maintaining accurate results.

#Report 3

Problem Information

Problem Title: Calculating π Using OpenMP

Lecture Number: Unit 2, Discussion 3

Problem Description

The goal of this program is to approximate the value of π using numerical integration. The interval is divided into many small parts, and the area under the curve is calculated to estimate π .

Parallelization Approach

Using an atomic to remove impact of false sharing.

Solution Explanation

1. Divide the interval into small steps.
2. Create multiple threads using OpenMP.
3. Each thread calculates part of the sum.
4. Combine all partial sums to compute π .

The program uses an atomic operation to safely update the shared variable.

Source Code

```
E: > College > Parallel Computing > C++ atomic.cpp
1  #include <iostream>
2  #include <omp.h>
3  using namespace std;
4
5  const long NUM_STEPS = 100000;
6  const int NUM_THREADS = 2;
7
8  int main()
9  {
10     double pi = 0.0;
11     double step = 1.0 / NUM_STEPS;
12
13     #pragma omp parallel num_threads(NUM_THREADS)
14     {
15         int threadId = omp_get_thread_num();
16         int numThreads = omp_get_num_threads();
17
18         double sum = 0.0;
19
20         // Each thread works on a portion of the iterations
21         for (long i = threadId; i < NUM_STEPS; i += numThreads)
22         {
23             double x = (i + 0.5) * step;
24             sum += 4.0 / (1.0 + x * x);
25         }
26
27         // Calculate this thread's contribution
28         sum *= step;
29
30         // Add it to the final result safely
31         #pragma omp atomic
32         pi += sum;
33     }
34
35     cout << "PI = " << pi << endl;
36
37     return 0;
38 }
```

Execution Results

```
PS E:\College\Parallel Computing> g++ -fopenmp atomic.cpp -o atomic
PS E:\College\Parallel Computing> ./atomic
PI = 3.14159
```

Conclusion

Using OpenMP allows the computation to run faster by sharing the work between threads. The program successfully approximates π while demonstrating basic parallel programming concepts.

#Report 4

Problem Information

Problem Title: Parallel Average Calculation Using OpenMP

Lecture Number: Unit 2: The core features of OpenMP - Mod 5: Parallel Loops (making the Pi program simple)

Problem Description

calculate the average value of a set of numbers using OpenMP parallel programming.

Parallelization Approach

The program uses OpenMP to parallelize the summation of array elements.

Each thread processes a portion of the array and computes a partial sum. OpenMP automatically combines all partial sums into a single shared variable using the reduction clause. This eliminates race conditions and ensures that the final result is computed correctly.

Solution Explanation

The program begins by defining an array of numerical values and initializing a variable to store the total sum. A parallel for loop is created using OpenMP. The reduction clause is applied to the accumulation variable so that each thread maintains its own local copy during execution.

After the summation is completed, the total sum is divided by the number of elements in the array to obtain the average value.

Finally, the average is displayed on the screen.

Source Code

```
#include <iostream>
#include <omp.h>
#define SIZE 5
using namespace std;
int main() {
    double result = 0.0, data[SIZE] = { 10,20,30,40,50 }; int k;
#pragma omp parallel for reduction(+:result)
    for (k = 0; k < SIZE; k++) result += data[k];
    result /= SIZE;
    cout<<"Average Value = "<< result;
    return 0;
}
```

Execution Results

```
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>g++ -fopenmp parallel.cpp
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>.\a.exe
Average Value = 30
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>
```

Conclusion

demonstrated the use of OpenMP reduction for parallel programming in C++. The array summation task was successfully parallelized using a parallel for loop with a reduction clause.

#Report 5

Problem Information

Problem Title: A multi-threaded "Hello world" program

Lecture Number: Unit 1: Getting started with OpenMP - Disc 1: Hello world and how threads work

Problem Description

Write a multithreaded program where each thread prints "hello world".

Parallelization Approach

The program uses OpenMP's parallel directive to create a team of threads. Once the parallel region starts, each thread executes the same block of code independently.

The function `omp_get_thread_num()` is used to identify the thread number of each thread

Solution Explanation

When the program starts, OpenMP creates multiple threads. Each thread obtains its unique ID and prints two messages containing that ID. Since all threads write to the console at the same time, the order of the output may differ from one execution to another.

Source Code

```
#include <iostream>
#include <omp.h>
using namespace std;
int main() {
#pragma omp parallel
{
    int threadid = omp_get_thread_num();
    cout << "Hello threadid = " << threadid << endl;
    cout << "Worldthreadid = " << threadid << endl;
}
return 0;
}
```

Execution Results

```
C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>g++ -fopenmp parallel.cpp

C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>.\a.exe
Hello threadid = Hello threadid = 1Hello threadid = 0
Worldthreadid = 0
Hello threadid = 6
Worldthreadid = Hello threadid = 4
Worldthreadid = 4
6
5
Worldthreadid = 5

Worldthreadid = 1
Hello threadid = 2
Worldthreadid = 2
Hello threadid = 7
Worldthreadid = 7
Hello threadid = 3
Worldthreadid = 3

C:\Users\DELL\Desktop\FCI4.2\network_programing\Apps\projectNP\shahd\parallel>|
```

Conclusion

This program demonstrates the basic concept of parallel programming using OpenMP. Multiple threads are created and executed simultaneously, with each thread printing its own ID.